

PROJECT TITLE: IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

1. Problem Statement

1.1 Core Problem Description

In standard IT Service Management (ITSM) workflows using ServiceNow, the default Incident creation and management interface often lacks dynamic, real-time validations. Because forms load statically with all fields exposed regardless of context, end-users and service desk agents frequently submit incomplete, inaccurate, or poorly categorized incident records.

1.2 Operational Impact

The absence of immediate client-side checks and dynamic form controls leads to several critical bottlenecks:

- **Data Integrity Issues:** Technicians receive tickets without proper descriptions, incorrect urgency levels, or missing error details, delaying initial triage.
- **Cognitive Overload:** Users are forced to navigate cluttered forms containing irrelevant fields that do not apply to their specific issue type.
- **Delayed Resolutions:** Every incorrect or incomplete submission requires back-and-forth communication between support staff and the reporter, directly increasing the Mean Time to Resolution (MTTR).

2. Detailed Breakdown of Pain Points

To clearly define the scope of our intervention, our team categorized the problem into three main technical domains within ServiceNow:

2.1 UI and Layout Clutter

- **Issue:** Standard forms show all fields by default, regardless of whether they are relevant to the specific incident category or subcategory.
- **Consequence:** Users get confused about which fields are mandatory for their specific problem type.

2.2 Lack of Pre-Submission Validation

- **Issue:** Users can click the "Submit" or "Save" button even when critical fields are empty or contain invalid data patterns.
- **Consequence:** Incomplete records are written directly to the database, requiring manual cleanup by administrators.

2.3 Inconsistent Urgency and Priority Assignment

- **Issue:** Users frequently select "High" or "Critical" urgency for minor issues, or conversely, mark major service outages as "Low."
- **Consequence:** Service desk queues become skewed, and critical business-impacting incidents get delayed.

3. Target Beneficiaries (Who is Affected?)

- **End-Users / Reporters:** Experience frustration due to confusing form layouts and lack of clear guidance while logging complaints.
- **Service Desk Agents:** Waste valuable time clarifying vague tickets, requesting additional information, and re-categorizing incorrectly filed incidents.
- **IT Managers & Administrators:** Struggle with inaccurate reporting metrics, poor dashboard analytics, and inefficient service delivery pipelines due to bad input data.

4. Proposed Problem Resolution Objective

The objective of our Naan Mudhalvan project is to systematically eliminate these defined problems by:

1. Utilizing **UI Policies** to dynamically manage field visibility, read-only states, and mandatory requirements without writing server-side code.
2. Implementing **Client Scripts** (onChange, onLoad, onSubmit) to intercept user actions, execute instant browser-level validations, and enforce strict data entry standards before any record reaches the database.

5. Conclusion

By precisely defining these operational bottlenecks, our project establishes a clear roadmap for building an optimized, automated, and user-friendly Incident management interface in ServiceNow. This problem definition directly drives the creation of our technical architecture and subsequent testing phases.